Progetto – Telemetria sicura per la catena del freddo

Realizzate un sistema client-server HTTP in cui sensori di temperatura inviano misure in JSON, cifrate con una chiave concordata tramite Diffie-Hellman e validate da uno JSON Schema. Il lavoro è **individuale**.

Scenario

FrigoTrack S.r.l. è un'azienda di logistica che trasporta alimenti freschi e farmaci termosensibili. Ogni cella frigorifera e ogni camion ha un sensore di temperatura e umidità che invia le misure a una piattaforma centrale. Oggi i dati viaggiano in chiaro su HTTP: un concorrente, o un tecnico malintenzionato, potrebbe leggerli o alterarli, e un lotto di farmaci "certificato" con dati falsi metterebbe l'azienda in grave difficoltà.

Vi viene affidato lo sviluppo di un **prototipo** che risolva il problema. Dovete consegnare un server di raccolta dati, un client che simula i sensori e la documentazione che permetta all'azienda di valutare la soluzione.

Il cliente pone tre vincoli:

- il canale resta **HTTP** (niente HTTPS): la cifratura è a carico dell'applicazione;
- la chiave di cifratura **non deve mai viaggiare in rete**, quindi va concordata con Diffie-Hellman;
- ogni messaggio deve avere un **formato rigoroso**, verificato con JSON Schema, perché i dati entrano in un sistema di tracciabilità che non accetta valori fuori standard.

Requisiti obbligatori

Server HTTP

1. Espone gli endpoint descritti nella sezione Protocollo.
2. Gestisce più client contemporanei, ciascuno con la propria sessione e la propria chiave.
3. Valida ogni messaggio in ingresso con JSON Schema e risponde con codici HTTP corretti (200, 400, 401, 404).
4. Salva le misure ricevute (file JSON) e le rende consultabili.

Client

1. Simula almeno 3 sensori con identificativo diverso, che inviano misure a intervalli regolari.
2. Esegue l'handshake e ricava la chiave prima di inviare qualsiasi dato.
3. Valida il messaggio con lo schema *prima* di cifrarlo.

Sicurezza

1. Scambio chiavi con **Diffie-Hellman** implementato dallo studente: calcolo di `A = g^a mod p` e del segreto condiviso `K = B^a mod p`. È ammesso usare le funzioni di potenza modulare del linguaggio, non librerie che eseguano l'intero scambio.
2. Il segreto condiviso si può usare direttamente come chiave simmetrica.
3. In rete non compare mai nessun dato in chiaro, tranne l'identificativo di sessione.

JSON Schema

1. Uno schema per ogni tipo di messaggio, in file separati nella cartella `schemas/`.
2. Uso di vincoli reali: `type`, `required`, `enum`, `minimum`/`maximum`, `pattern`, `format`, `additionalProperties: false`.
3. Un messaggio non valido viene rifiutato con un errore che indica il campo sbagliato.

Protocollo

Il dialogo ha due fasi: prima l'handshake, che produce la chiave di sessione, poi lo scambio di messaggi cifrati.

Endpoint	Metodo	Corpo	Risposta
`/api/handshake`	POST	`p`, `g`, chiave pubblica del client `A` (in chiaro)	`session_id`, chiave pubblica del server `B`
`/api/telemetry`	POST	busta cifrata con la misura del sensore	busta cifrata con conferma (`ack`) ed eventuale allarme
`/api/readings`	POST	busta cifrata con la richiesta (sensore, intervallo)	busta cifrata con l'elenco delle misure
`/api/close`	POST	busta cifrata	chiusura sessione e cancellazione della chiave

Passi dell'handshake:

1. Il client sceglie `p` e `g` (gruppo noto, per esempio RFC 3526) e un segreto casuale `a`; invia `A = g^a mod p`.
2. Il server sceglie `b`, calcola `B = g^b mod p`, genera il `session_id` e risponde con `B`.
3. Entrambi calcolano `K = g^(ab) mod p` e derivano la chiave.

Ogni messaggio successivo viaggia in una **busta** con campi `session_id`, `nonce`, `ciphertext` e `tag` (codificati in Base64). All'interno della busta c'è il JSON applicativo, che il ricevente decifra e valida con lo schema.

La regola di sicurezza da rispettare nel codice è questa: il server decifra, *poi* valida; se la decifratura o il tag falliscono, risponde 400 senza elaborare nulla.

Il server genera un allarme nella risposta quando la temperatura esce dall'intervallo di sicurezza del tipo di merce (per esempio farmaci tra 2 e 8 °C).

Esempio di messaggio e schema

Messaggio applicativo che il sensore invia (prima di essere cifrato):

```
{
  "sensor_id": "CELLA-BG-007",
  "timestamp": "2026-11-02T08:15:30Z",
  "cargo_type": "pharma",
  "temperature_c": 4.6,
  "humidity": 55,
  "battery": 87
}
```

Schema corrispondente (JSON Schema draft 2020-12), da prendere come **punto di partenza** e da completare:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Misura sensore",
  "type": "object",
  "required": ["sensor_id", "timestamp", "cargo_type", "temperature_c"],
  "properties": {
    "sensor_id": { "type": "string", "pattern": "^[A-Z]+-[A-Z]{2}-[0-9]{3}$" },
    "timestamp": { "type": "string", "format": "date-time" },
    "cargo_type": { "enum": ["food", "pharma", "frozen"] },
    "temperature_c": { "type": "number", "minimum": -40, "maximum": 40 },
    "humidity": { "type": "number", "minimum": 0, "maximum": 100 },
    "battery": { "type": "integer", "minimum": 0, "maximum": 100 }
  },
  "additionalProperties": false
}
```

Busta cifrata effettivamente trasmessa in rete:

```
{
  "session_id": "a3f9c2e1",
  "ciphertext": "mH3t...Base64...==",
  "tag": "8dK2...Base64...=="
}
```

Gli studenti devono scrivere gli schemi per la richiesta `readings`, per la sua risposta e per il messaggio di errore.

Consegne e piano di lavoro

Da consegnare in un repository Git (con cronologia dei commit leggibile):

- codice di server e client, avviabili con pochi comandi;
- cartella `schemas/` con tutti gli schemi;
- `README.md` con istruzioni di avvio, diagramma di sequenza dell'handshake e descrizione dei messaggi;
- almeno 5 test (handshake riuscito, messaggio valido, messaggio non valido, busta manomessa, sessione inesistente);
- una pagina di relazione: scelte tecniche e limiti noti della soluzione.

Valutazione e interrogazione pratica

Il progetto viene valutato in un'interrogazione pratica individuale di circa 30 minuti. Il lavoro è individuale: ogni studente sviluppa il proprio progetto e deve saper spiegare e modificare qualunque parte del proprio codice.

Criterio	Peso
Qualità del progetto	25%
Interrogazione: risposte alle domande	25%
Modifica dal vivo	50%

Svolgimento dell'interrogazione:

1. **Demo**: avvio di server e client, handshake, invio di una misura valida e di una non valida.
2. **Domande**.
3. **Modifica dal vivo**: lo studente applica una variazione richiesta dal docente.

Per evitare lavori copiati, la modifica dal vivo e le domande sul codice hanno un peso diretto sul voto, e la cronologia dei commit viene consultata durante la prova.